

# Plantilla multi-formato para investigación aplicada en buenas prácticas de desarrollo de software

Autor Uno\*, Autor Dos†

\*Afilación 1, Ciudad, País — [autor1@ejemplo.edu](mailto:autor1@ejemplo.edu)

†Afilación 2, Ciudad, País — [autor2@ejemplo.edu](mailto:autor2@ejemplo.edu)

## Resumen—

### RESUMEN

La arquitectura por capas ha sido, por décadas, el patrón estructural más influyente en la ingeniería de software. Su valor reside en la capacidad de dividir sistemas complejos en componentes manejables, donde cada capa asume responsabilidades bien definidas. Sin embargo, el entorno actual —caracterizado por la adopción de la inteligencia artificial, el Internet de las Cosas, la computación en la nube y los microservicios— ha puesto a prueba los límites de este paradigma.

El presente trabajo analiza la evolución técnica de la arquitectura por capas a partir de veinte estudios académicos contemporáneos, con el objetivo de identificar su impacto práctico en la mantenibilidad, rendimiento, escalabilidad y testabilidad de los sistemas modernos. La investigación sintetiza cómo los avances en teoría de grafos, visualización arquitectónica, modelado de rendimiento y refactorización automatizada han permitido mantener la relevancia del patrón, al tiempo que revelan la necesidad de adaptarlo a nuevos contextos como el aprendizaje automático y las arquitecturas distribuidas.

Los resultados demuestran que, aunque la arquitectura por capas conserva su vigencia conceptual, su aplicación práctica requiere una comprensión dinámica del entorno operativo y un enfoque flexible de diseño. Más que un patrón rígido, hoy representa un marco adaptable capaz de integrarse con estrategias como la Arquitectura Limpia, los pipelines autocontenidos y las capas especializadas para modelos de aprendizaje automático.

**Palabras clave:** arquitectura por capas, mantenibilidad, microservicios, rendimiento, refactorización, aprendizaje automático.

**Index Terms—**buenas prácticas, ingeniería de software, investigación aplicada, reproducibilidad

### INTRODUCCIÓN

El crecimiento constante de la complejidad en los sistemas de software ha generado la necesidad de estructuras arquitectónicas que promuevan orden, claridad y mantenibilidad. Desde sus primeras formulaciones, la arquitectura por capas ha servido como un modelo de referencia para alcanzar dichos objetivos. En ella, las responsabilidades se distribuyen jerárquicamente, permitiendo que las capas superiores dependan de los servicios de las inferiores sin violar la separación de preocupaciones. Sin embargo, el paso del tiempo y la evolución de los paradigmas tecnológicos han planteado nuevos desafíos a este patrón.

Diversos estudios, como los analizados en esta investigación, han abordado tanto las limitaciones como las oportunidades de la arquitectura por capas en entornos modernos. En particular, los avances en sistemas distribuidos inteligentes, aprendizaje automático y aplicaciones basadas en microservicios han exigido replantear los mecanismos de comunicación, despliegue y prueba dentro de los sistemas. El patrón clásico, aunque sólido conceptualmente, no siempre satisface los requisitos de rendimiento y flexibilidad demandados por las arquitecturas contemporáneas.

La motivación central de este trabajo surge de esa brecha entre el valor teórico de la arquitectura por capas y su aplicación práctica en contextos tecnológicos emergentes. Los artículos revisados muestran intentos por redefinir sus métricas, mejorar su testabilidad, introducir capas adicionales para seguridad o procesamiento, e incluso hibridar el patrón con nuevos enfoques como la Arquitectura Limpia y los pipelines autocontenidos.

El objetivo general de esta investigación es analizar, desde una perspectiva técnica y aplicada, cómo ha evolucionado la arquitectura por capas y cuáles son los impactos reales de su adopción en proyectos modernos. Para lograrlo, se desarrolló una revisión analítica y comparativa de veinte artículos académicos que abordan esta temática desde distintas perspectivas: teórica, metodológica y empírica.

De esta manera, el presente artículo busca aportar una comprensión actualizada sobre el papel de la arquitectura por capas en el desarrollo de software contemporáneo, destacando su relevancia, sus limitaciones y las tendencias que orientan su transformación hacia modelos más flexibles, escalables y sostenibles.

### TRABAJOS RELACIONADOS

La arquitectura por capas ha sido abordada desde múltiples perspectivas en la literatura contemporánea. Los estudios más recientes evidencian una transición desde su concepción clásica hacia modelos más adaptativos, enfocados en la automatización, el rendimiento y la integración con tecnologías emergentes. Esta sección presenta una síntesis de los principales aportes encontrados en veinte artículos relevantes.

En primer lugar, investigaciones como las de Belle et al. (2021) y Tu (2023) confirman la persistencia del patrón en la práctica industrial, destacando su valor en la

organización modular del software. Sin embargo, también señalan la fragilidad del modelo cuando se enfrenta a sistemas de gran escala, en los cuales las violaciones de capas o *skip-calls* erosionan la mantenibilidad del código. Este fenómeno, descrito como “erosión arquitectónica”, ha sido analizado por Li et al. (2022), quienes argumentan que sus causas no siempre son técnicas, sino también organizacionales.

Desde un enfoque de automatización, Schmidt et al. (2014) y Kobayashi et al. (2013) proponen métodos de reconstrucción arquitectónica mediante técnicas de *clustering* y visualización. Estos trabajos demuestran que la arquitectura puede ser redescubierta a partir de las dependencias reales del código, lo cual es esencial para el mantenimiento de sistemas heredados. Complementariamente, Thakare y Kiwelekar (2021) plantean la necesidad de redefinir las métricas de evaluación, priorizando la detección de violaciones estructurales sobre las métricas clásicas de acoplamiento y cohesión.

En cuanto a la aplicación de la arquitectura por capas en entornos especializados, Darwish (2016) introduce un modelo extendido para el Internet de las Cosas (IoT), añadiendo capas de seguridad y middleware. De forma similar, Bastidas Fuertes y Pérez Hernández (2023) aplican el patrón a entornos multi-lenguaje mediante el uso de *transpilers*, buscando mantener la lógica de negocio consistente entre distintas plataformas.

Los enfoques modernos también incluyen la integración de arquitecturas por capas en sistemas de inteligencia artificial. Zhang et al. (2024) proponen una estructura de tres niveles —modelo, inferencia y aplicación— para mejorar la escalabilidad y el control en sistemas basados en modelos de lenguaje extensos (LLMs). Muccini y Vaidhyanathan (2021) complementan esta idea al resaltar la necesidad de que la arquitectura contemple tanto los componentes de software como los elementos de datos y aprendizaje.

En el ámbito del rendimiento, Sharma, Jalote y Trivedi (2005) desarrollan un modelo matemático basado en cadenas de Markov para estimar la latencia y el rendimiento de sistemas por capas. Este enfoque permite predecir cuellos de botella sin necesidad de implementar el sistema completo. A su vez, Meijer, Trubiani y Aleti (2024) evalúan empíricamente patrones arquitectónicos en microservicios, confirmando que las predicciones teóricas de rendimiento pueden alinearse con los resultados experimentales bajo condiciones controladas.

Otros estudios relevantes, como los de Contreras Rivas et al. (2025) y Patwardhan et al. (2016), demuestran cómo la estructura por capas sigue siendo aplicable incluso en escenarios de emergencia o despliegue continuo, siempre que se combine con estrategias de independencia de módulos y segmentación funcional. Brown et al. (2021), por su parte, ejemplifican el éxito de la transición hacia Arquitectura Limpia mediante la refactorización de un sistema educativo inteligente, mostrando que los principios de separación de dependencias pueden revitalizar sistemas

complejos.

Finalmente, Shahin et al. (2018) enfatizan la importancia de la arquitectura como habilitador de la entrega continua, mientras que Misell (2023) recuerda que los aspectos de seguridad estructural y de comunicación deben coexistir para garantizar la integridad del sistema.

En conjunto, estos estudios confirman que la arquitectura por capas continúa siendo un fundamento esencial del diseño de software moderno, pero su aplicación debe evolucionar hacia estructuras más flexibles, medibles y conscientes del contexto operativo en el que se desempeñan.

## METODOLOGÍA

El presente estudio se desarrolló bajo un enfoque cualitativo y analítico, sustentado en la revisión documental de veinte artículos académicos publicados entre 2005 y 2025. Estos trabajos fueron seleccionados por su relevancia en torno al tema de la arquitectura por capas, su evolución conceptual y las aplicaciones prácticas en entornos modernos de desarrollo de software.

### *Selección de fuentes*

Se utilizaron criterios de inclusión basados en tres factores: actualidad, aplicabilidad técnica y diversidad metodológica. Los artículos analizados incluyen propuestas teóricas, estudios empíricos, revisiones sistemáticas y casos de implementación. Las fuentes fueron obtenidas de bases académicas reconocidas como IEEE, Springer, Elsevier y arXiv, garantizando su rigor científico.

### *Estrategia de análisis*

El análisis se desarrolló en tres etapas complementarias:

1. **Clasificación temática:** Los artículos se agruparon en categorías según su enfoque principal: (a) definición y evolución del patrón por capas, (b) medición y reconstrucción arquitectónica, (c) aplicación del patrón en nuevos contextos (IoT, ML, LLMs, microservicios), y (d) evaluación del rendimiento y la mantenibilidad.
2. **Comparación de enfoques:** Se identificaron similitudes y divergencias entre las propuestas, considerando dimensiones como objetivos, técnicas empleadas, resultados obtenidos y limitaciones señaladas por los autores.
3. **Síntesis interpretativa:** A partir del contraste de resultados, se elaboró una interpretación general del impacto práctico del patrón en la ingeniería de software actual, destacando las tendencias más significativas.

### *Criterios de evaluación*

Para asegurar la consistencia del análisis, se aplicaron criterios técnicos derivados de la literatura revisada, tales como:

- **Mantenibilidad:** Grado en que la arquitectura facilita el cambio y la extensión del sistema.

- **Escalabilidad:** Capacidad del diseño para crecer en funcionalidad o volumen de datos sin comprometer el rendimiento.
- **Testabilidad:** Posibilidad de aislar componentes y validar su comportamiento de forma independiente.
- **Rendimiento:** Medición del impacto que la estructura en capas tiene sobre la latencia y el throughput del sistema.
- **Sostenibilidad estructural:** Resistencia de la arquitectura frente a la erosión o degradación por violaciones de dependencia.

#### *Síntesis metodológica*

La integración de los datos se realizó mediante un enfoque narrativo comparativo, priorizando la interpretación cualitativa sobre la cuantificación estadística. El objetivo no fue determinar una arquitectura “ideal”, sino comprender cómo los principios del modelo por capas se adaptan a los entornos tecnológicos actuales. Este proceso permitió establecer una visión integral sobre las transformaciones, limitaciones y oportunidades prácticas que enfrenta el patrón en la actualidad.

#### METODOLOGÍA

El presente estudio se desarrolló bajo un enfoque cualitativo y analítico, sustentado en la revisión documental de veinte artículos académicos publicados entre 2005 y 2025. Estos trabajos fueron seleccionados por su relevancia en torno al tema de la arquitectura por capas, su evolución conceptual y las aplicaciones prácticas en entornos modernos de desarrollo de software.

#### *Selección de fuentes*

Se utilizaron criterios de inclusión basados en tres factores: actualidad, aplicabilidad técnica y diversidad metodológica. Los artículos analizados incluyen propuestas teóricas, estudios empíricos, revisiones sistemáticas y casos de implementación. Las fuentes fueron obtenidas de bases académicas reconocidas como IEEE, Springer, Elsevier y arXiv, garantizando su rigor científico.

#### *Estrategia de análisis*

El análisis se desarrolló en tres etapas complementarias:

1. **Clasificación temática:** Los artículos se agruparon en categorías según su enfoque principal: (a) definición y evolución del patrón por capas, (b) medición y reconstrucción arquitectónica, (c) aplicación del patrón en nuevos contextos (IoT, ML, LLMs, microservicios), y (d) evaluación del rendimiento y la mantenibilidad.
2. **Comparación de enfoques:** Se identificaron similitudes y divergencias entre las propuestas, considerando dimensiones como objetivos, técnicas empleadas, resultados obtenidos y limitaciones señaladas por los autores.
3. **Síntesis interpretativa:** A partir del contraste de resultados, se elaboró una interpretación general del

impacto práctico del patrón en la ingeniería de software actual, destacando las tendencias más significativas.

#### *Criterios de evaluación*

Para asegurar la consistencia del análisis, se aplicaron criterios técnicos derivados de la literatura revisada, tales como:

- **Mantenibilidad:** Grado en que la arquitectura facilita el cambio y la extensión del sistema.
- **Escalabilidad:** Capacidad del diseño para crecer en funcionalidad o volumen de datos sin comprometer el rendimiento.
- **Testabilidad:** Posibilidad de aislar componentes y validar su comportamiento de forma independiente.
- **Rendimiento:** Medición del impacto que la estructura en capas tiene sobre la latencia y el throughput del sistema.
- **Sostenibilidad estructural:** Resistencia de la arquitectura frente a la erosión o degradación por violaciones de dependencia.

#### *Síntesis metodológica*

La integración de los datos se realizó mediante un enfoque narrativo comparativo, priorizando la interpretación cualitativa sobre la cuantificación estadística. El objetivo no fue determinar una arquitectura “ideal”, sino comprender cómo los principios del modelo por capas se adaptan a los entornos tecnológicos actuales. Este proceso permitió establecer una visión integral sobre las transformaciones, limitaciones y oportunidades prácticas que enfrenta el patrón en la actualidad.

#### RESULTADOS

El análisis comparativo de los veinte artículos permitió identificar una serie de tendencias y hallazgos que ilustran la evolución práctica de la arquitectura por capas y su papel dentro de la ingeniería de software moderna. Los resultados se organizan en torno a cuatro ejes principales: adaptabilidad, rendimiento, mantenibilidad y sostenibilidad estructural.

#### *1. Adaptabilidad del patrón*

La primera tendencia observada es la adaptación del modelo por capas a distintos entornos tecnológicos. En sistemas de IoT, como señala Darwish (2016), la arquitectura se ha extendido a cinco o más capas para incluir funciones de seguridad, middleware y gestión de datos. En contraste, Zhang et al. (2024) demuestran que los sistemas basados en aprendizaje automático adoptan una versión especializada de tres capas enfocadas en el flujo de inferencia. Esto evidencia que el patrón conserva su relevancia al reconfigurarse según las necesidades del dominio.

## 2. Rendimiento y eficiencia

El rendimiento continúa siendo uno de los aspectos más estudiados. Sharma, Jalote y Trivedi (2005) muestran que las estructuras jerárquicas bien equilibradas mejoran el throughput y reducen la latencia, aunque a costa de una complejidad analítica mayor. Meijer, Trubiani y Aleti (2024) confirman empíricamente que los patrones de diseño aplicados a microservicios —como Pipes and Filters— pueden optimizar el rendimiento sin sacrificar la modularidad. Estos resultados sugieren que la arquitectura por capas, complementada con técnicas de segmentación y paralelismo, puede alcanzar niveles de eficiencia comparables a los de los modelos no estructurados.

## 3. Mantenibilidad y refactorización

Una tercera observación relevante es la importancia de la automatización en el mantenimiento arquitectónico. Schmidt et al. (2014) y Brown et al. (2021) demuestran que los sistemas legados pueden recuperar su estructura original mediante refactorización guiada por métricas o reglas de dependencia. Este tipo de intervención reduce el costo de mantenimiento y facilita la integración de nuevas funcionalidades sin alterar la lógica principal. Thakare y Kiwelekar (2021) añaden que la evaluación continua de violaciones de capa constituye una métrica de salud estructural más efectiva que las medidas tradicionales de calidad de código.

## 4. Sostenibilidad estructural y erosión

El concepto de erosión arquitectónica, descrito por Li et al. (2022), es fundamental para entender los desafíos actuales. La pérdida de conformidad entre la arquitectura conceptual y la implementación real afecta la testabilidad y la estabilidad del sistema. Los estudios revisados coinciden en que la sostenibilidad estructural depende tanto de la disciplina organizacional como de las herramientas técnicas. La automatización, la documentación activa y la visualización de dependencias son estrategias recurrentes para mitigar este riesgo.

## Síntesis general

En conjunto, los resultados confirman que la arquitectura por capas sigue siendo un paradigma vigente, pero en constante reinvencción. Su impacto práctico radica en su flexibilidad para adaptarse a nuevos contextos tecnológicos, su capacidad de integración con arquitecturas emergentes y su potencial para ser medida y controlada de forma automatizada. Los hallazgos refuerzan la idea de que el éxito de este patrón no depende únicamente de su diseño inicial, sino de su gestión evolutiva a lo largo del ciclo de vida del software.

## DISCUSIÓN

El análisis de los trabajos revisados permite comprender que la arquitectura por capas, más que un modelo fijo, constituye un marco de pensamiento adaptable que continúa guiando el desarrollo de software contemporáneo. Sin

embargo, su efectividad depende de cómo se interpreta y se implementa en función del contexto tecnológico y organizacional.

## 1. Relevancia y vigencia del patrón

A pesar de la aparición de nuevos estilos arquitectónicos —como microservicios, pipelines autocontenidos y arquitecturas limpias—, la estructura por capas mantiene su pertinencia conceptual. Su principio de separación de responsabilidades sigue siendo esencial para garantizar la modularidad y la testabilidad. Tal como plantean Belle et al. (2021) y Tu (2023), la fortaleza del patrón reside en su simplicidad, pero su debilidad se manifiesta cuando la disciplina de las dependencias se pierde durante el mantenimiento.

## 2. Impacto en la práctica profesional

En los entornos de desarrollo actuales, caracterizados por la integración continua y los ciclos ágiles, la arquitectura por capas actúa como una referencia organizacional más que como una regla estricta. Estudios como los de Shahin et al. (2018) y Patwardhan et al. (2016) destacan que los equipos que logran desacoplar sus capas operativas alcanzan una mayor capacidad de entrega continua, reduciendo los riesgos de regresión. En contraste, la falta de separación clara de capas conduce a una degradación progresiva de la calidad estructural, fenómeno ampliamente documentado como erosión arquitectónica.

## 3. Limitaciones observadas

Aunque las capas ofrecen claridad estructural, también imponen una sobrecarga en la comunicación entre módulos. Sharma, Jalote y Trivedi (2005) advierten que la latencia inherente al flujo secuencial de peticiones puede ser crítica en sistemas de alta concurrencia. Por su parte, Darwish (2016) y Contreras Rivas et al. (2025) señalan que la ampliación del modelo con nuevas capas —como seguridad o middleware— mejora la funcionalidad pero puede reducir la eficiencia global si no se gestiona adecuadamente la complejidad.

## 4. Hacia un modelo híbrido

Los resultados indican que la tendencia más sólida es la hibridación del patrón por capas con otros enfoques. Los modelos propuestos por Bastidas Fuertes y Pérez Hernández (2023) y Brown et al. (2021) evidencian que las capas pueden coexistir con estructuras limpias o con generadores automáticos de código sin perder sus principios básicos. De igual modo, la introducción de capas específicas para aprendizaje automático o inferencia (Zhang et al., 2024) demuestra que el patrón puede evolucionar sin desarticular su esencia.

## 5. Implicaciones futuras

La revisión sugiere que el futuro de la arquitectura por capas no depende tanto de su rediseño conceptual, sino de su integración con la automatización, la medición continua y las prácticas de ingeniería de software sostenible. Las herramientas de visualización, las métricas de conformidad estructural y las técnicas de refactorización automatizada emergen como los pilares que garantizarán la longevidad del patrón en entornos cambiantes.

En conclusión, la arquitectura por capas continúa siendo una herramienta fundamental para la ingeniería de software moderna. Su vigencia no radica en su rigidez, sino en su capacidad para adaptarse a nuevas realidades técnicas sin renunciar a sus principios fundacionales de orden, modularidad y separación de responsabilidades.

## CONCLUSIONES

El estudio permitió constatar que la arquitectura por capas sigue siendo un patrón fundamental dentro de la ingeniería de software, no por su rigidez, sino por su capacidad de adaptación a contextos tecnológicos diversos. Su persistencia en el tiempo demuestra que los principios de modularidad, separación de responsabilidades y jerarquía funcional continúan siendo esenciales para el desarrollo de sistemas sostenibles.

En primer lugar, los resultados evidencian que el patrón por capas ha evolucionado hacia modelos híbridos, en los que convive con paradigmas como la Arquitectura Limpia, los microservicios y los pipelines autocontenidos. Esta integración refuerza la autonomía de los módulos y mejora la escalabilidad sin perder la coherencia conceptual del diseño.

En segundo lugar, la literatura analizada revela que la mantenibilidad y la sostenibilidad arquitectónica son los desafíos más críticos. La erosión estructural, causada por violaciones de dependencias y falta de disciplina organizacional, puede neutralizar las ventajas del patrón. En consecuencia, la adopción de herramientas de análisis estático, visualización y refactorización automatizada se consolida como una práctica necesaria para prolongar la vida útil de los sistemas.

Asimismo, se comprobó que el rendimiento no debe considerarse una limitación inherente a las capas, sino una variable controlable mediante una adecuada distribución de responsabilidades y la aplicación de modelos de evaluación temprana. La incorporación de técnicas de modelado, como las redes de colas o los algoritmos de clustering, permite anticipar cuellos de botella y optimizar la estructura antes de su implementación.

Finalmente, el impacto práctico del patrón por capas trasciende la ingeniería técnica: influye directamente en la organización del trabajo, la comunicación entre equipos y la capacidad de evolución del software. Su futuro dependerá de la convergencia entre diseño arquitectónico, automatización inteligente y sostenibilidad estructural. En este sentido, la arquitectura por capas continúa siendo no

solo una referencia histórica, sino una herramienta vigente para enfrentar los retos de la complejidad creciente en el desarrollo de software.

## REFERENCIAS

- Bastidas Fuertes, A. F., & Pérez Hernández, M. G. (2023). *Transpiler-Based Architecture Design Model for Back-End Layers in Software Development*. Applied Sciences, 13(20), 11371.
- Belle, A. B., El Boussaidi, G., Lethbridge, T. C., Kpodjedo, S., Mili, H., & Paz, A. (2021). *Systematically reviewing the layered architectural pattern principles and their use to reconstruct software architectures*. arXiv preprint arXiv:2112.01644.
- Brown, P. S., Dimitrova, V., Hart, G., Cohn, A. G., & Moura, P. O. (2021). *Refactoring the Whitby Intelligent Tutoring System for Clean Architecture*. Theory and Practice of Logic Programming, 21(6), 818–834.
- Contreras Rivas, L. Y., López Domínguez, E., Hernández Velázquez, Y., Domínguez Isidro, S., Medina Nieto, M. A., & De La Calleja, J. (2025). *A Layered Software Architecture for the Development of Smart Mobile Distributed Systems Oriented to the Management of Emergency Cases*. Applied Sciences, 15(7), 3664.
- Darwish, M. (2016). *Improved layered architecture for Internet of Things*. Middle East Africa Computing Science & Engineering, 2(1), 1–13.
- Kobayashi, K., Kamimura, M., Yano, K., Kato, K., & Matsuo, A. (2013). *SARF Map: Visualizing Software Architecture from Feature and Layer Viewpoints*. In 2013 21st IEEE International Conference on Program Comprehension (ICPC) (pp. 43–52). IEEE.
- Li, R., Liang, P., Soliman, M., & Avgeriou, P. (2022). *Understanding software architecture erosion: A systematic mapping study*. Journal of Software: Evolution and Process, 34(6), e2423.
- Llisterri Giménez, N., Monfort Grau, M., & Freitag, F. (2022). *On-device training of machine learning models on microcontrollers with federated learning*. Electronics, 11(4), 573.
- Meijer, W., Trubiani, C., & Aleti, A. (2024). *Experimental evaluation of architectural software performance design patterns in microservices*. Journal of Systems and Software, 212, Article 112183.
- Misell, Q. (2023). *Hiding in text/plain sight: Security defences of Tor Onion Services*. arXiv preprint arXiv:2312.00545.
- Muccini, H., & Vaidhyanathan, K. (2021). *Software Architecture for ML-based Systems: What Exists and What Lies Ahead*. In Proceedings of the 1st International Workshop on AI Engineering - Software Engineering for AI (WAIN) (pp. 53–60). IEEE/ACM.
- Patwardhan, A., Patwardhan, R., & Vartak, S. (2016). *Self-Contained Cross-Cutting Pipeline Software Archi-*

*tecture*. International Research Journal of Engineering and Technology, 3(5), 1858–1861.

- Schmidt, F., MacDonell, S. G., & Connor, A. M. (2014). *An automatic architecture reconstruction and refactoring framework*. Software Engineering Research, Management and Applications, 377, 95–111.
- Shahin, M., Zahedi, M., Babar, M. A., & Zhu, L. (2018). *An empirical study of architecting for continuous delivery and deployment*. Empirical Software Engineering, 23(6), 3332–3372.
- Sharma, V. S., Jalote, P., & Trivedi, K. S. (2005). *Evaluating performance attributes of layered software architecture*. Lecture Notes in Computer Science, 377, 333–348.
- Thakare, S. B., & Kiwelekar, A. (2021). *Redefining measures of Layered Architecture*. arXiv preprint arXiv:2106.03079.
- Tu, Z. (2023). *Research on the Application of Layered Architecture in Computer Software Development*. Journal of Computing and Electronic Information Management, 11(3), 34–38.
- Zhang, D., Xu, X., Wang, C., Xing, Z., & Mao, R. (2024). *A Layered Architecture for Developing and Enhancing Capabilities in Large Language Model-based Software Systems*. arXiv preprint arXiv:2411.12357.
- Belle, A. B., et al. (2021). *Systematically reviewing the layered architectural pattern principles and their use to reconstruct software architectures*. arXiv preprint arXiv:2112.01644.
- Contreras Rivas, L. Y., et al. (2025). *A Layered Software Architecture for the Development of Smart Mobile Distributed Systems Oriented to the Management of Emergency Cases*. Applied Sciences, 15(7), 3664.